NAME: ALAN PIOSAM A

REG NO:411625104004

DEPARTMENT: COMPUTER AND SCIENCE ENGINEERING

SUBJECT: PYTHON

BATCH: 3

TOPIC:Treasure Hunt mini game

Aim:

To create a simple Treasure Hunt mini game in Python using list, dictionary, tuple, set, loops, and conditional statements.

Procedure:

Step 1: Import Module

1. Import the `random` module to randomly select the treasure location.

Step 2: Initialize Data

2. Create a **list** called `locations` containing:
   - Forest
   - Cave
   - River
   - Temple
   - Village
3. Create a **dictionary** called `clues` that stores a clue for each location.
4. Create another **dictionary** called `coordinates` that stores coordinate values (x, y) for each location.

Step 3: Select Treasure Location

5. Use `random.choice(locations)` to randomly select one location.
6. Store it in a variable called `treasure_location`.

Step 4: Create Visited Tracker

7. Create an empty **set** called `visited_places` to store searched locations.

Step 5: Display Welcome Message

8. Print:
    - Welcome message
    - Game instructions

Step 6: Start Game Loop

9. Start an infinite loop using `while True:`.

Step 7: Show Available Locations

10. Display all locations from the `locations` list.

Step 8: Take User Input

11. Ask the player to enter a location.
12. Store input in variable `choice`.

Step 9: Validate Input

13. If `choice` is NOT in `locations`:

- Print "Invalid location"
- Continue to next loop iteration.

Step 10: Check Already Visited

14. If `choice` is already in `visited_places`:

- Print "Already searched"
- Continue to next loop iteration.

Step 11: Mark Location as Visited

15. Add `choice` to `visited_places` set.

Step 12: Display Clue & Coordinates

16. Print:

- Clue from `clues[choice]`
- Coordinates from `coordinates[choice]`

Step 13: Check Treasure Condition

17. If `choice == treasure_location`:

- Print Congratulations message
- Break the loop (end game)

18. Else:

- Print "No treasure here"
- Continue loop

Step 14: End Game

19. After loop ends:

- Print visited places
- Print "Game Over"

CODE:

```
import random
```

```
# List of locations (Map)
locations = ["Forest", "Cave", "River", "Temple", "Village"]

# Dictionary with clues
clues = {
    "Forest": "Lots of trees around...",
    "Cave": "It's dark and cold...",
    "River": "Water is flowing...",
    "Temple": "Ancient and peaceful place...",
    "Village": "People are living here..."
}

# Dictionary with coordinates (tuple)
coordinates = {
    "Forest": (2, 3),
    "Cave": (5, 1),
    "River": (1, 4),
    "Temple": (3, 5),
    "Village": (4, 2)
}

# Random treasure location
treasure_location = random.choice(locations)
```

```
# Set to store visited places
visited_places = set()

print("🏴‍☠️ Welcome to Treasure Hunt Game!")
print("Find the hidden treasure!\n")

while True:
    print("\nAvailable Locations:")
    for place in locations:
        print("-", place)

    choice = input("\nEnter location to search: ")

    # Condition check
    if choice not in locations:
        print("❌ Invalid location! Try again.")
        continue

    if choice in visited_places:
        print("⚠️ You already searched here!")
        continue

    visited_places.add(choice)

    print("\nClue:", clues[choice])
```

```
    print("Coordinates:", coordinates[choice])

    if choice == treasure_location:
        print("\n🎉 Congratulations! You found the treasure!")
        break
    else:
        print("No treasure here. Keep searching!")

print("\nVisited Places:", visited_places)
print("Game Over!")
```

Output:

```
Output                                                    Clear
🏴‍☠️ Welcome to Treasure Hunt Game!
Find the hidden treasure!


Available Locations:
- Forest
- Cave
- River
- Temple
- Village

Enter location to search: River

Clue: Water is flowing...
Coordinates: (1, 4)
No treasure here. Keep searching!
```

RESULT:

The Treasure Hunt Mini Game was successfully developed using Python